AZURE DATABRICKS

Comprehensive Practical Assessment

City Water Network Monitoring and Leak Detection

55 hands-on exercises | SQL + PySpark + Delta Lake + Lakeflow | No marks or solutions

Scenario

You are supporting a municipal water utility. Smart meters send flow, pressure and water-quality measurements from distribution zones. The operations team also tracks service requests and capacity changes. Your goal is to clean and analyze the data, manage Delta tables, process incremental records, and publish actionable analytics.

Datasets provided

Dataset	Rows	Purpose
water_meter_readings.csv	157	Initial meter readings, including quality issues
water_zones.json	11	Zone master with nested location fields
water_service_requests.csv	36	Operational maintenance requests
water_reading_changes.csv	12	Existing and new readings for MERGE
water_readings_incremental.csv	18	New append-only readings for pipeline refresh

Getting started

• Create or select a Unity Catalog schema and a writable Volume. Upload the five dataset files to a folder in the Volume.

• Import databricks_setup.py as a notebook, configure its widgets and run it once to create the Delta tables.

• Do not rerun setup after beginning the exercises. Work on table copies for UPDATE, DELETE and MERGE operations.

• Use a Lakeflow pipeline for SQL CREATE OR REFRESH STREAMING TABLE and Python pyspark.pipelines definitions.

Data quality note

The data deliberately contains duplicates, null values, invalid measurements, an unknown zone, and a zone without readings. Investigate these rather than assuming all input records are valid.

Section A | SQL Querying and Filtering

01. Inspect water_meter_readings_source and display its schema and total row count.
02. Retrieve reading_id, meter_id, zone_id, reading_time and consumption_liters for readings marked Valid.
03. Find readings where pressure_bar is below 2.5, ordering the results from lowest pressure upward.
04. Find records where flow_lpm is between 20 and 50 and the source_system is Gateway or SmartMeter.
05. List distinct reading_status values and count how many readings belong to each status.

Section B | Joins, Aggregations and Window Functions

06. Join meter readings with water_zones and show zone_name, district, meter_id and consumption_liters.
07. Find reading records whose zone_id has no matching entry in water_zones.
08. Find zones that currently have no meter readings, using a suitable join.
09. Calculate total valid consumption, average pressure and reading count for each zone; exclude nonpositive consumption.
10. Rank meters by total valid consumption within each district using a SQL window function.

Section C | PySpark DataFrame Operations

11. Load the meter readings and zone master Delta tables into separate PySpark DataFrames; print their schemas.
12. Use select(), filter() and orderBy() to display high-flow Valid readings.
13. Add a flow_category column: High for flow_lpm >= 60, Medium for 30 to below 60, Low otherwise; handle nulls explicitly.
14. Join the DataFrames and calculate average flow_lpm for each zone_type.
15. Use PySpark groupBy() and agg() to identify the three zones with the highest valid water consumption.

Section D | Data Quality and Cleaning

16. Identify duplicate reading_id values and display every row associated with those IDs.
17. Find missing pressure_bar and source_system values and count the affected records.
18. Identify invalid measurements: negative flow_lpm, nonpositive consumption_liters or missing zone_id.
19. Create water_readings_clean as a new Delta table containing one row per reading_id, with Valid status and positive measurements; document the duplicate-selection rule.
20. Use the service request data to identify zones with High-priority requests and compare their average recorded pressure.

Section E | Delta Lake Tables and CRUD

21. Create water_readings_work_copy using CREATE TABLE AS SELECT from water_meter_readings_work.
22. Insert two new readings into water_readings_work_copy using SQL; use new unique reading IDs.
23. Update the status of a chosen Suspect reading in the copy to Rejected and verify the change.
24. Delete Rejected readings from the copy and compare counts before and after deletion.

25. Use PySpark to create a Delta table named zone_consumption_summary from an aggregated DataFrame.

Section F | MERGE, UPSERT and SCD

26. Compare water_reading_changes with water_meter_readings_work and separate matching IDs from new IDs.
27. Run a SQL MERGE on water_meter_readings_work using reading_id as the key: update matches and insert nonmatches.
28. Validate that changed reading IDs have new values and that new IDs were inserted.
29. Perform the same SCD Type 1 MERGE using the PySpark DeltaTable API on a fresh working-table copy.
30. Build a zone_capacity_history SCD Type 2 table from water_zones. Effective 2026-10-20, change W02 capacity to 195000 and W06 to 305000, retaining old versions and current flags.

Section G | Delta History, Time Travel and Maintenance

31. Use DESCRIBE DETAIL on water_meter_readings_work to inspect the format, storage location, file count and size.
32. Use DESCRIBE HISTORY to identify recorded table operations and locate the version produced by MERGE.
33. Query the version immediately before the MERGE using VERSION AS OF; compare reading 1005 with its current value.
34. Execute OPTIMIZE on a suitable working Delta table; compare its file count before and after and explain the result.
35. Explain the purpose of VACUUM and inspect eligible files with a dry run where supported; do not delete retained historical files.

Section H | Unity Catalog, Views and Parquet

36. Identify the catalog and schema containing your assessment tables; display the managed Delta table location.
37. Create a permanent view for valid readings with zone names and districts.
38. Create a session-scoped temporary view for low-pressure readings and query it.
39. Create and query a global temporary view of High-priority service requests if your compute supports it; explain its scope.
40. Write a DataFrame to Parquet in an authorized Unity Catalog Volume, then load that Parquet dataset into a new managed Delta table.

Section I | Structured Streaming

41. Create a streaming DataFrame using the built-in rate source with five rows per second.
42. Transform the generated value into a simulated meter_id and flow_lpm column.
43. Filter the stream to retain simulated flow_lpm above a threshold of your choice.

44. Aggregate streaming records by a derived flow category and display the live counts using a supported sink.

45. Stop the running streaming query and explain the role of a checkpoint for durable streaming output.

Section J | Lakeflow / Delta Live Tables

46. Create a SQL Lakeflow pipeline with a Bronze streaming table that reads append-only water_meter_readings_source.

47. Create a Silver streaming table filtering Valid readings with positive flow and consumption.

48. Create a Gold materialized view aggregating total consumption, average pressure and reading counts by zone.

49. Run the SQL pipeline; inspect its dependency graph and verify Bronze, Silver and Gold outputs.

50. Create a separate Python Lakeflow pipeline using pyspark.pipelines to implement equivalent Bronze, Silver and Gold tables with different names.

Section K | Incremental Loads, Analytics and Visualizations

51. Inspect water_readings_incremental_staging and append its records to water_meter_readings_source without overwriting existing source data.

52. Rerun the SQL Lakeflow pipeline and verify that the Gold results reflect the newly appended readings.

53. Build a bar chart showing total valid consumption by zone and a pie chart showing consumption share by district.

54. Build a line chart showing daily total consumption and a scatter plot of pressure_bar versus flow_lpm for valid readings.

55. Create a final dashboard or notebook report combining zone consumption, water-quality indicators and service request counts; explain two operational findings.

Final capstone | Water Operations Analytics

• Build a Bronze-Silver-Gold Lakeflow solution from the supplied meter readings and zone master.

• Publish clean readings and per-zone and per-district summaries; flag potential low-pressure and water-quality concerns.

• Demonstrate one MERGE, one historical Delta query, an incremental pipeline update, and four visualizations.

• Submit your notebooks, pipeline definitions, pipeline graph screenshot, and chart screenshots.

Environment notes

Some streaming, global temporary view and maintenance commands require supported compute. Use approved Unity Catalog Volumes for file operations. Do not directly modify managed Delta table storage paths. No solutions are provided.